Laboratorio di Calcolo per Fisici

Esercitazione n. 5, 9-10 Novembre

Canale A-Dec 2026-2027, Docenti: G. Artoni, L. Caprini

Lo scopo della quinta esercitazione di laboratorio è di fare pratica con le istruzioni di controllo di flusso `if...(then)...else`; `while...do`, le funzioni di generazione di numeri casuali (*random*) e gli array, scrivendo un programma che simuli il gioco della roulette.

Fate il login utilizzando `lcgaXXX` sia come *userid* sia come *password*, dove `XXX` è il codice a tre cifre associato al vostro gruppo, così come indicato nella comunicazione dei gruppi.
In quanto segue, si assume che la vostra **`home`** sia `/home/studenti/lcgaXXX`. L'indirizzo potrebbe essere diverso, ma potete controllarlo immediatamente aprendo il terminale ed utilizzando il comando `pwd`.

1. Aprite una finestra di *terminale*.
2. Nella vostra home create una cartella chiamata `EX5` e posizionatevi all'interno di quest'ultima. Tutti i file, gli script e i codici C dovranno essere contenuti all'interno di questa cartella.

▶ Prima parte

Scrivete un programma `lancio.c` che, utilizzando opportunamente le funzioni di generazione di numeri casuali, simuli una mano di una partita di roulette (lancio e risultato). Il programma deve:

1. Generare un numero casuale intero X compreso tra 1 e 36 (incluso).
2. Riconoscere se il numero generato X è pari o dispari e se è $X \leq 18$ oppure $X > 18$.
   - Se il numero è pari verrà stampata la lettera E (*even*)
   - Se il numero è dispari verrà stampata la lettera O (*odd*)
   - Se il numero è minore o uguale a 18 verrà stampata la lettera M (*manque*)
   - Se il numero è maggiore di 18 verrà stampata la lettera P (*passe*)
3. Stampare il risultato nel formato: `X E/O M/P` (cioè numero estratto pari (o dispari) minore (o maggiore) di 18). Esempio: `12 E M`.

Suggerimento: Ricordiamo prima di tutto come si generano dei numeri interi casuali con i generatori di numeri random del C. Le librerie standard del C dispongono di diverse funzioni per la generazione di numeri casuali. La funzione `lrand48()` di `stdlib.h` restituisce un numero intero casuale compreso tra `[0,RAND_MAX]`. Per generare un numero **intero** in un intervallo generico [**`min, max`**] e assegnarlo ad una variabile intera si può usare sia `lrand48()` che `drand48()`:

```
int rand_1 = lrand48() % (max - min + 1) + min;
int rand_2 = (int)(drand48() * (max - min + 1)) + min;
```

Nota Bene: ricordatevi di inizializzare il generatore di numeri random usando, una sola volta all'inizio del vostro programma, la funzione `srand48(seed)`, dove `seed` è una variabile intera positiva che contiene il "seme" del generatore. `seed` può essere un numero qualsiasi definito nel programma, ad esempio, se `min = 20` e `max = 30`,

```c
#include <stdlib.h>

int main(){
  int seed = 3498723;
  srand48(seed);
  int min = 20, max = 30;
  // due modi equivalenti per estrarre un numero random in [min, max]
  int rand_1 = drand48() * (max - min + 1) + min; // <- il metodo che vi
    consiglio
  int rand_2 = lrand48() % (max - min + 1) + min;
}
```

Per utilizzare queste funzioni si deve includere il file d'intestazione `stdlib.h`. In questo modo, dato che il valore di `seed` è sempre lo stesso, il numero estratto con `lrand48()` sarà sempre lo stesso. Per capire il motivo rileggete le note del corso sull'algoritmo lineare congruenziale.
Alternativamente, per inizializzare il seme si può usare la funzione `time()` con argomento `NULL`, che restituisce il tempo in secondi a partire dal 1/1/1970. In questo caso la sequenza di numeri random sarà diversa a ogni esecuzione del codice. Per usare la funzione `time` bisogna includere il file d'intestazione `time.h`:

```c
#include <stdlib.h>
#include <time.h>

int main(){
  int seed = time(NULL);
  srand48(seed);
  int min = 20, max = 30;
  int rand_1 = drand48() * (max - min + 1) + min;
  int rand_2 = lrand48() % (max - min + 1) + min;
}
```

► **Seconda parte**

Copiate il programma `lancio.c` in `roulette.c` (usando il comando linux opportuno). Modificate `roulette.c` in modo che, invece che un singolo lancio, il programma simuli $N \geq 100$ lanci, dove N è una costante simbolica il cui valore verrà modificato in fase di compilazione usando l'istruzione `gcc -o roulette.x roulette.c -DN=YYY` (dove `YYY` è il numero di lanci).

1. Alla fine degli N lanci, stampate sullo schermo la *frequenza* di ciascun risultato (E/O/M/P), cioè

il numero di volte in cui si è verificato un risultato diviso il numero di lanci totali: la frequenza è quella attesa o si discosta dal valore aspettato? Verificate come varia la frequenza al variare del parametro N.

Suggerimento: Per generare gli `N` numeri random dovrete utilizzare un ciclo iterativo, come ad esempio un ciclo `for()`;

```
...
for(i = 0; i < N; i++){
  int rand_num = drand48() * (max - min + 1) + min;
  ...
}
...
```

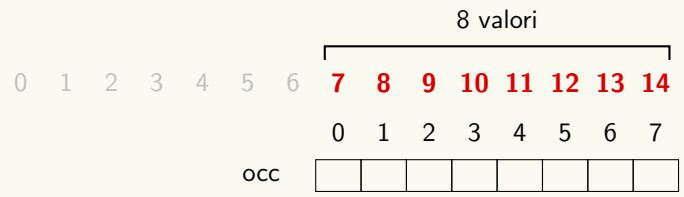
All'interno del ciclo vanno inseriti i controlli sul tipo di numero estratto. Definite le variabili che contino quante volte il numero uscito è pari/dispari minore/maggiore di 18, ricordando di inizializzarle a zero.

```
// inizializzazione dei contatori
...
for(i = 0; i < N; i++){
  int rand_num = drand48() * (max - min + 1) + min;
  // se rand_num e' dispari incrementate di 1 il contatore dei dispari,
    altrimenti quello dei pari
  // se rand_num e' > 18 incrementate di 1 il contatore dei numeri > 18,
    altrimenti quello dei <= 18
  ...
}
....
```

2. Costruite un *array* delle occorrenze che conti quante volte è uscito ciascun numero tra 1 e 36.

Suggerimento: L'array delle occorrenze conta quante volte un certo numero è stato estratto. Per comodità chiamiamo l'array `occ`: ricordatevi di inizializzarlo a 0! Tale array dovrà contenere un'informazione per tutti i possibili numeri compresi nell'intervallo `[min, max]`, quindi la sua lunghezza sarà pari a `(max - min + 1)`: per convincervi provate a contare quanti numeri interi sono compresi in $[0, 10]$ (estremi inclusi) e verificate se questo valore corrisponde a `(10 - 0 + 1)`. Ogni volta che si estrae un numero random, bisogna incrementare di 1 il contenuto della casella di memoria di `occ` associata a quel numero. Per capire qual è la casella di memoria associata si osservi il seguente esempio. Supponiamo di voler estrarre numeri random nell'intervallo $[7, 14]$. Per quanto detto prima l'array `occ` dovrà essere di lunghezza 8

```
....
int min = 7, max = 14;
int occ[8] = {0}; // inizializziamo tutte le caselle a zero
....
```



La figura qui sopra mostra, nella riga superiore in rosso, i numeri che possono essere estratti nell'intervallo $[7, 14]$ e nella riga inferiore l'array `occ`. Ogni volta che viene estratto il 7 si deve incrementare di 1 la casella di memoria 0 di `occ`, `occ[0]`, che conta proprio le volte che il numero 7 viene estratto. Analogamente, ogni volta che viene estratto l'8 viene incrementato di 1 `occ[1]`, se viene estratto il 9 viene incrementato di 1 `occ[2]`, ecc., fino al caso in cui venga estratto il 14, che corrisponde a `occ[7]`. La casella di memoria associata ad un generico numero random estratto `x` è quindi `occ[x-7]`. Generalizzando questo ragionamento si può scrivere, ad esempio, il seguente ciclo `for()`:

```
....
int i;
int min = 7, max = 14;
int occ[8] = {0};
...
for(i = 0; i < N; i++){
  int x = drand48() * (max - min + 1) + min;
  occ[x - min]++; // oppure occ[x - min] += 1;
}
....
```

3. Scrivete il contenuto dell'array in un file `isto.dat` che contenga su tre colonne i numeri da uno a 36 (detti anche *bin*), le relative occorrenze e le frequenze (cioè le occorrenze divise per il numero di lanci). Questi dati serviranno per costruire un *istogramma* dei risultati. Il file può essere creato a mano, o redirigendo l'output del programma su un file con il comando `./roulette.x > isto.dat`. Ricordate di usare `fprintf(stderr, ...)` per redirigere eventuali istruzioni fornite dal programma all'utente a schermo ed evitare che finiscano anch'esse nel file di output.

Suggerimento: dopo che tutti gli `N` numeri sono stati estratti, usate un ciclo `for` che va da `[0, max - min + 1)` per stampare il numero estratto, le occorrenze e le frequenze

associate. Il ciclo può essere del tipo:

```
for(i = 0; i < (max - min + 1); i++){
  printf("...", i + min, occ[i], occ[i] / (double)N);
}
```

oppure

```
for(i = min; i < (max + 1); i++){
  printf("...", i, occ[i - min], occ[i - min] / (double)N);
}
```

Notate che per il calcolo delle frequenze è stato usato l'operatore di cast `(double)` applicato ad `N` per evitare di effettuare una divisione tra interi.

► **Terza parte**

Fate girare il programma `roulette.c` con un numero variabile di lanci, ed effettuate un'analisi dell'andamento dell'istogramma dei risultati.

1. Create con python un istogramma della distribuzione dei lanci per un numero N fissato di lanci ($N \geq 100$) e salvatelo su un file `isto1.png`. Per creare un grafico a barre con python a partire da due liste `x` e `y` (contenenti il numero del bin e il numero di occorrenze) si usa il comando `plt.bar(x, y, fill=True, color='...', label='...')` al posto di `plt.plot(x, y, ...)`.

2. Generate un certo numero di file di nome `istoxx.dat` (`isto10.dat`, `isto100.dat`, ecc.) che contengano l'istogramma dei risultati generati dal programma `roulette.c`, per `xx` lanci diversi: 10, 100, 10.000, 100.000. Per confrontare esecuzioni del programma (*run*) con numeri diversi di lanci, dovete utilizzare la *frequenza* con cui si è verificato ciascun risultato e non il numero di occorrenze.

3. Formattando opportunamente il grafico, confrontate gli istogrammi ottenuti per diversi valori di N. Che cosa si può dire sulla distribuzione dei risultati in funzione di N? Il risultato è coerente con le vostre aspettative? Se no, perché?

► **Quarta parte (facoltativa)**

A partire dal programma `roulette.c` create un nuovo programma `gioco.c` che simuli una vera partita di roulette. Una partita si svolge come segue:

1. All'inizio del gioco, il giocatore riceve una dotazione iniziale di 100 euro.

2. A ogni mano, il giocatore può decidere di fare una puntata su pari o dispari, o su un numero maggiore o minore di 18 (Manque/Passe). In caso di vincita, il giocatore riceve due volte la posta per le puntate Pari/Dispari o Manque/Passe.

3. Il gioco termina dopo un numero fissato di mani (10 o 20) che il giocatore sceglie prima di iniziare a giocare, oppure quando il giocatore esaurisce il credito a sua disposizione.

Il vostro programma dovrà simulare tutte le fasi della partita; a ogni mano dovrà chiedere al giocatore di effettuare una puntata, verificare l'eventuale vittoria e controllare se il giocatore abbia ancora soldi a propria disposizione per giocare una nuova mano. Alla fine della partita, il programma dovrà stampare un messaggio riassuntivo che riporti il numero totale delle mani giocate, l'ammontare totale delle puntate, e il credito a disposizione del giocatore.